Chapitre IV :
Introduction au langage VHDL

1- Définition

- Le langage **VHDL (Very high speed circuit Hardware Description Language)** est un Langage de description matérielle utilisé pour modéliser, simuler et synthétiser des circuits numériques.

2- Structure d'un code VHDL

❖ Déclaration des bibliothèques

- Le programme VHDL commence par la déclaration de la bibliothèque IEEE par utilisation de la commande library IEEE
- La bibliothèque IEEE contient l'ensemble de paquetages utilisés pour décrire les composants logiques.
- Parmi les paquetage indispensable pour les composants logiques, on trouve: STD_LOGIC_1164; STD_LOGIC_UNSIGNED; STD_LOGIC_ARITH; NUMERIC_STD…
- Pour utiliser un paquetage, on utilise le mot clé « use »
- Pour utiliser l'intégralité d'un paquetage on utilise le suffixe «.all »

❑ Exemple de déclaration d'une bibliothèque

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

2- Structure d'un code VHDL

❖ Déclaration de l'entité

- Une entité définit les entrées et les sorties d'un composant

❑ Syntaxe d'une entité

```
entity nom de l'entité is
Port ( description des signaux);
end nom de l'entité;
```

❖ Déclaration de l'architecture

- L'architecture décrit le fonctionnement interne d'un circuit.

❑ Syntaxe d'une architecture

```
architecture nom de l'architecture of nom de l'entité is
   -- Signaux et constantes internes
   -- Fonctions internes
    begin
   -- Instructions
   end nom de l'architecture;
```

2- Structure d'un code VHDL

- Exemple: Description VHDL d'une porte logique AND




```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

Déclaration de la bibliothèque

```
entity AND is
    Port ( E1 : in STD_LOGIC;
           E2 : in STD_LOGIC;
            S :  out STD_LOGIC);
end AND;
```

Déclaration de l'entité

```
architecture Behavioral of AND is
begin
    S <= E1 and E2;
    end Behavioral;
```

Déclaration de l'architecture

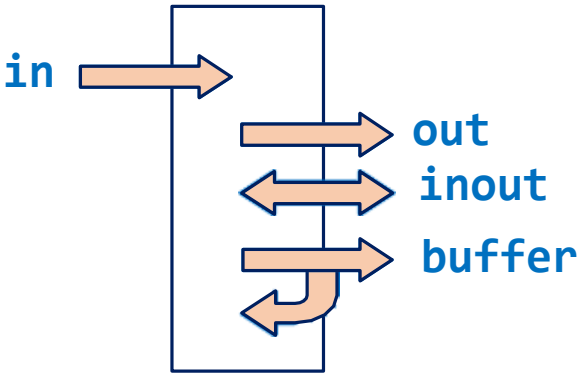
3- Signaux

- Le nom d'un signal est composé de caractères, le premier caractère doit être une lettre.
- Les lettres majuscules et minuscules sont équivalentes.

❑ Sens du signal

- Pour définir les signaux internes dans une entité, il y a quatre sens:

  - **`in:`** pour les entrées,
  - **`out`**: pour les sorties,
  - **`inout:`** pour les signaux bidirectionnels,
  - **`buffer`** : pour les signaux de sortie réentrants.



❑ Types

- Il y a plusieurs types de signaux d'entrées/sorties

  - **Bit**: deux valeurs : '0' ou '1'. Ce type est peu utilisé en synthèse et simulation.
  - **`Boolean`**: deux valeurs possibles : true ou false

3- Signaux

❑ Types

- **`STD_LOGIC:`**
  - Trois valeurs ayant un sens physique
    - **`'0'`**, **`'1'`** et **`'Z'`** pour la haute impédance.
  - Six valeurs utilisées en simulation :
    - **`'U'`** pour non initialisé,
    - **`'X'`** pour un résultat inconnu,
    - **`'L'`** pour un niveau bas probable,
    - **`'H'`** pour un niveau haut probable,
    - **`'W'`** niveau non qualifiable par 'L' ou 'H',
    - **`'-'`** sans importance.

- **`Types logiques vectoriels`**:
  - Un vecteur de Bit défini par: **Bit_VECTOR**
  - Un vecteur de STD_LOGIC défini par: **STD_LOGIC_VECTOR**
  - Déclaration par: **STD_LOGIC_VECTOR(N-1 downto 0)** ou **STD_LOGIC_VECTOR(0 to N-1)**

3- Signaux

❑ Types

- **Exemple:**
  - A : in std_logic_vector(3 downto 0)
    A(3) désigne le poids fort et A(0) est le poids faible

A(3)	A(2)	A(1)	A(0)

  - A : in std_logic_vecto (0 to 3)
    A(0) désigne le poids fort et A(3) est le poids faible

A(0)	A(1)	A(2)	A(3)

❑ Types mathématiques

- **INTEGER:** Type entier de référence (32 bits par défaut et limité à 32 bits sur certains simulateurs).
  - Mot clé **range**: Permet de spécifier l'intervalle des valeurs d'un INTEGER au moment de sa déclaration.
    - Exemple: a: integer range 0 to 10; -- b est un entier limité à l'intervalle [0;10]
- **NATURAL :** est un sous-type d'INTEGER. C'est un entier positif ou nul
- **POSITIVE:** est un sous-type d'INTEGER. C'est un entier positif
- **REAL:**Type réel en virgule flottante simple précision à la norme IEEE754. (ce type est utilisé avec le paquetage use **ieee.math_real.all**;)

3- Signaux

❑ Types caractères et chaînes de caractères

- CHARCTER: exemple ‘a’
- STRING: chaine de caractère exemple ‘’Erreur’’

❑ Déclaration d’une constante

- En VHDL, une constante est une valeur fixe, connue à la compilation, qui ne peut jamais changer pendant la simulation ou l’exécution.

constant NOM_CONSTANTE : **TYPE** := valeur;

- TYPE peut être : integer, natural, real, std_logic, std_logic_vector, etc.valeur est fixe et ne peut pas être modifiée après la déclaration.

Exemple

```
constant taille_bus : integer := 8;

constant  C : std_logic_vector(3 downto 0) := "1010";
```

3- Signaux

❑ Conversion de type

- STD_LOGIC_VECTOR est un simple vecteur de STD_LOGIC. Donc, aucune information sur la donnée stockée.
- Pour effectuer des opérations arithmétiques, il faut passer par des types **SIGNED** ou **UNSIGNED** qui sont aussi des vecteurs de STD_LOGIC.
- Le paquetage de référence qui définit ces types est: **NUMERIC_STD (use IEEE.NUMERIC_STD.ALL;)**
- UNSIGNED: Vecteur binaire représentant un entier naturel positif.
- SIGNED : Vecteur binaire représentant un entier signé (positif ou négatif) représenté en complément à deux.
- La déclaration **use IEEE.NUMERIC_STD.ALL;** permet :
  - La définition des types SIGNED et UNSIGNED,
  - Les conversions entre INTEGER et SIGNED ou UNSIGNED.
  - La définition des opérateurs relationnels et arithmétiques pour les SIGNED et UNSIGNED, entre INTEGER et SIGNED ou UNSIGNED.

3- Signaux

❑ Conversion de type

❖ Conversions importantes

- std_logic_vector vers signed ou unsigned

signed(vect_y);
unsigned(vect_y);

- Unsigned ou signed vers std_logic_vector

std_logic_vector(unsigned_signal)
std_logic_vector(signed_signal)

- De std_logic_vector vers entire:

to_integer(unsigned(vect_y))
to_integer(signed(vect_y))

exemple

to_integer(unsigned("00001010")) **--→ 10**
to_integer(signed("11111101")) **---→ -3**

- Entier vers std_logic_vector

std_logic_vector(to_unsigned(10, 8)) **-- 10 codé sur 8 bits----------→ "00001010"**
std_logic_vector(to_signed(-3, 8)) **-- -3 codé sur 8 bits----------→ "11111101"**

3- Signaux

❑ Déclaration des signaux

- Un signal externe (entité) dans **port,** la syntaxe est déclaré par :

NOM_DU_SIGNAL : sens type;

- Un signal interne à une architecture est déclaré par:

signal NOM_DU_SIGNAL : type;

- **Exemples:**
  - **signal A : std_logic;**
  - Déclaration multiple de signaux**: signal a,b,c : std_logic;**
  - Déclaration et initialisation d'un signal logique: signal a : std_logic := '1';
  - Déclaration d'un vecteur de signaux logiques: signal a : std_logic_vector (3 downto 0);
  - Déclaration et initialisation d'un vecteur de signaux logiques:

signal a : std_logic_vector (3 downto 0) := ''0000'';

  - Déclaration d'un signal integer: signal a : INTEGER range 0 to 1023;

3- Signaux

N.B: - Quand on a un seul bit on le met entre cote (ex: '0', '1')
- Plus d'un bit on le met entre guillemets (''): ex '' 0011''

- Les vecteurs de bits:
  - Représentation binaire "1010"
  - Représentation octale O"12"
  - Représentation hexadécimale X"A"

4- Les opérateurs

❑ Affectation simple

- Cet opérateur permet de modifier l'état d'un signal en fonction d'autres signaux et/ou d'autres opérateurs.
- **Exemple:**

```
S1 <= not(A);
S2 <= A and B;
S3 <= A or B;
S4 <= A xor B;
```

4- Les opérateurs

❑ Opérateurs logiques

- Opérateur logiques de base : and , or , xor, not, nand, nor, xnor.
- Opérateur de décalage à gauche : SLL
- Opérateur de décalage à droite: SRL
- Opérateur de rotation à gauche ROL (rotation de la gauche vers la droite)
- Opérateur de rotation à droite ROR (rotation de la droite vers la gauche)

Exemple :

```
A <= "1010011"

B <= A SLL 2          B <= 1001100
C <= A SRL 3          C <= 0001010
D <= A ROL 4          D <= 0111010
E <= A ROR 2          E <= 1110100
```

4- Les opérateurs

❑ Opérateurs arithmétiques

- Ces opérateurs sont utiliés pour les types `INTEGER`, `SIGNED` et `UNSIGNED`)
  - + , - , * , /, `mod` (modulo), `rem` (reste signé).

Exemple

```
signal A, B : unsigned(3 downto 0);
signal S    : unsigned(3 downto 0);
S <= A + B;
```

❑ Opérateurs relationnels

- Ces opérateurs sont utilisés pour les types `INTEGER`, `SIGNED` et `UNSIGNED`)
  - < , >, <= , >= , = , /= (différent de).

❑ Opérateur de concaténation: &

Exemple : A <= “101";
B <= “001";
C <= A & B --------→ C <= 101001

4- Les opérateurs

- Exemple : description VHDL des portes logiques de base

```
library ieee;
use ieee.std_logic_1164.all;

-- déclaration de l'entité
entity portes_logiques is
port( A,B: in std_logic;
      S1, S2, S3, S4, S5, S6:out std_logic);
end portes_logiques;

-- description de l'architecture
architecture description of portes_logiques is
begin
S1 <= not(A);
S2 <= A and B;
S3 <= A or B;
S4 <= A xor B;
S4 <= A nand B;
S5 <= A nor B;
end description;
```

Commentaires

4- Les opérateurs

❑ Exemple : description VHDL d'un additionneur complet

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity add_complet is
   Port (
      A ,B,Cin   : in  STD_LOGIC;
      S,Cout     : out STD_LOGIC
);
end add_complet;
```

```
architecture description of add_complet is
begin
   S    <= A xor B xor Cin;
   Cout <= (A and B) or (Cin and (A xor B));
end description;
```



4- Les opérateurs

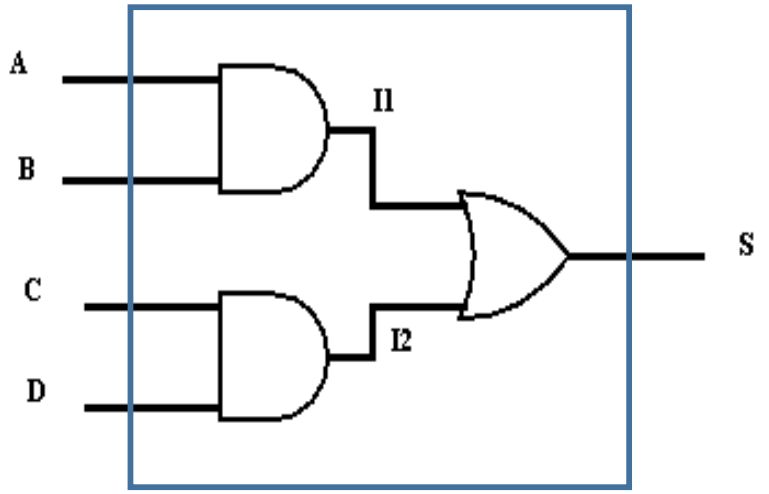
- Exemple : description VHDL d'un circuit logique avec signaux internes

```
library ieee;
use ieee.std_logic_1164.all;

-- déclaration de l'entité
entity circuit is
port( A,B,C,D: in std_logic;
      S:out std_logic);
end circuit;

-- description de l'architecture
architecture description of circuit is
-- Déclaration des signaux internes
signal I1,I2: std_logic;
begin
       I1 <= A and B;
       I2 <= C and D;
        S <= I1 or I2;

end description;
```



I1 et I2 sont des signaux internes: on doit les définir à l'intérieur de l'architecture

5- Domaine concurrent

- Les instructions sont exécutées en parallèle, l'ordre n'a pas d'importance.
- Ce sont des équations logiques, des affectations de signaux, des instanciations de composants, etc.
- Situé à l'intérieur d'une architecture, mais en dehors des process (voir domaine séquentiel).

❑ Instructions du domaine concurrent

- ❖ Affectation simple comme vu précédemment.
- ❖ Affectation conditionnelle:

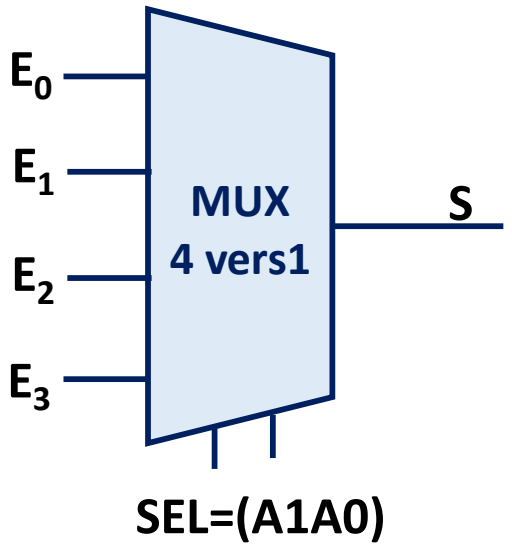
```
Signal <= expression when condition
else expression when condition
else expression when condition
else expression ;
```

Remarque : la dernière ligne "else expression" n'est pas obligatoire mais elle est fortement conseillée.

5- Domaine concurrent

- Exemple d'affectation conditionnelle: multiplexeur quatre vers un.

```
Library ieee;
USE iee.std_logic_1164.all;
Entity mux is
Port ( E0,E1,E2,E3 : in std_logic;
SEL : in std_logic_vector (1 downto 0);
S : out std_logic );
End mux;
Architecture description of mux is
Begin
S <= E0 when SEL = "00"
Else E1 when SEL = "01"
Else E2 when SEL = "10"
Else E3;
End description;
```



5- Domaine concurrent

❖ Affectation sélective: permet d'affecter différentes valeurs à un signal selon les valeurs prises par un signal de sélection.

```
With signal_de_selection select
signal <= expression when valeur_de_selection,
expression when valeur_de_selection,
expression when others;
```

❑ Exemple d'affectation sélective: multiplexeur quatre vers un.

```
Library ieee;
USE iee.std_logic_1164.all;
Entity mux is
Port ( E0,E1,E2,E3 : in std_logic;
SEL : in std_logic_vector (1 downto 0);
S : out std_logic );
End mux;
```

```
Architecture description of mux is
Begin
with SEL select
   S <= E0 when "00" ,
        E1 when "01",
        E2 when "10 ",
        E3 when "11",
        'X' when others;
End description;
```

6- Domaine séquentiel

- Le domaine séquentiel regroupe toutes les instructions qui s'exécutent :à l'intérieur d'un process, d'une fonction, ou d'une procedure dans un ordre précis, comme un programme classique.
- Contrairement au domaine concurrent (exécuté en parallèle), ici l'ordre des instructions compte.

❑ Le process

- Un process est un bloc qui contient des instructions séquentielles.
- Il permet de décrire du comportement, comme dans un langage de programmation.
- L'exécution d'un process est déclenché par un ou des changements d'états de signaux logiques. Le nom de ces signaux est défini dans la **liste de sensibilité** lors de la déclaration du process.
- Les process peuvent décrire aussi des circuits combinatoires.

```
[nom_du_process :] Process ( liste_de_sensibilite )
Begin
… instruction_du_process … ;
End process;
```

- Le nom du process est facultatif.

6- Domaine séquentiel

❑ Les variables

- Une variable ne peut être déclarée et utilisée que dans un process.
  - Déclaration avant le begin de du process.

```
process(liste_de_sensibilite)
   variable nom_variable : type_variable := valeur_initiale;
begin
```

  - Une variable n'existe qu'à l'intérieur d'un process et disparait au moment de la sortie du process.
  - La variable n'a pas un sens physique.
  - Le comportement d'une variable est différent des signaux, dont l'affectation est programmée pour la fin du process.
  - L'affectation d'une variable est instantanée.

Exemple

```
X := 5;
Y := X + 3;   -- ici Y vaut 8 immédiatement
```

6- Domaine séquentiel

❑ Les instructions du domaine séquentiel

- Ces instructions sont utilisées à l'intérieur d'un process

❖ Affectation simple

❖ Instruction de choix avec « if »

```
if condition then
   -- actions du then
end if;
```

```
if condition then
   -- actions du then
else
   -- actions du else
end if;
```

```
if condition then
   -- actions du then
elsif condition then
   -- actions du elsif then
else
   -- actions du elsif else
end if;
```

6- Domaine séquentiel

- Exemples de description VHDL des circuits combinatoires par utilisation de process
- Exemple : description VHDL d'un multiplexeur 4 vers 1 par utilisation d'un process et « if »

- **Avec if….. Then…elsif….**

```
Library ieee;
USE ieee.std_logic_1164.all;
Entity mux is
   Port (
      E0, E1, E2, E3 : in  std_logic;
      SEL : in  std_logic_vector (1 downto 0);
      S: out std_logic
   );
End mux;
```

```
Architecture comportement of mux is
Begin
   process(E0, E1, E2, E3, SEL)
   begin
      if SEL = "00" then
         S <= E0;
      elsif SEL = "01" then
         S <= E1;
      elsif SEL = "10" then
         S <= E2;
      elsif SEL = "11" then
         S <= E3;
      else  -- Facultative
         S <= 'X'; -- si sel n'est ni à 1 ni à 0, on affecte X à S
      end if;
   end process;
End comportement;
```

6- Domaine séquentiel

- Exemples de description VHDL des circuits combinatoires par utilisation de process
- Exemple : description VHDL d'un multiplexeur 4 vers 1 par utilisation d'un process et « if »

- **Avec if….. then**

```
Library ieee;
USE ieee.std_logic_1164.all;
Entity mux is
   Port (
      E0, E1, E2, E3 : in  std_logic;
      SEL : in  std_logic_vector (1 downto 0);
      S: out std_logic
   );
End mux;
```

```
Architecture comportement of mux is
Begin
   process(E0, E1, E2, E3, SEL)
   begin
      -- valeur pour les autres états
      S <= 'X';
      if SEL = "00" then
         S <= E0;
      end if;
      if SEL = "01" then
         S <= E1;
      end if;
      if SEL = "10" then
         S <= E2;
      end if;
      if SEL = "11" then
         S <= E3;
      end if;
   end process;
End comportement;
```

6- Domaine séquentiel

❑ Les instructions du domaine séquentiel

❖ Instruction « case »

- Cette instruction permet le choix entre plusieurs actions.
- Elle utilise aussi l'instruction WHEN.
- En fin de liste, on peut ajouter WHEN OTHERS qui permet de donner une action à tous les choix qui n'ont pu être trouvés dans la liste.

```
case signal de sélection is
       when valeur de sélection 1 =>  Instruction;
       when valeur de sélection 2 => Instruction;
        .
        .
        .
        when others => Instruction;
        end case;
```

- On peut combiner dans un même process les instructions « if » et « case ».

6- Domaine séquentiel

- Exemple : description VHDL d'un multiplexeur 4 vers 1 par utilisation d'un process et « Case »

```
Library ieee;
USE ieee.std_logic_1164.all;
Entity mux is
   Port (
      E0, E1, E2, E3 : in  std_logic;
      SEL : in  std_logic_vector (1 downto 0);
      S: out std_logic
   );
End mux;
```

```
Architecture comportement of mux is
Begin
   process(E0, E1, E2, E3, SEL)
   begin
      case SEL is
         when "00" => S <= E0;
         when "01" => S <= E1;
         when "10" => S <= E2;
          when "11" =>  S <= E3;
          when others =>   S <= 'X'; --ou bien S<='0';
end case;
   end process;
End comportement;
```

6- Domaine séquentiel

❑ Les instructions du domaine séquentiel

❖ Instruction « for…loop »

- Cette une instruction répétitive de plusieurs actions.

```
FOR parametre IN intervalle (Ex : 0 to 7)
LOOP
instruction;
End LOOP;
```

❖ Instruction « After»

- AFTER spécifie un temps de réponse d'un signal par rapport à son événement.

```
signal_name <=  expression AFTER time_expression
```

Exemple

```
S <= A and B AFTER 10 ns;          -- temps de propagation est de 10ns
```

6- Domaine séquentiel

- Exemple : description VHDL d'un mux 4 vers 1 par utilisation d'un process et « for…loop »

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity mux is
   port(
      E   : in  std_logic_vector(3 downto 0);
      SEL : in  std_logic_vector(1 downto 0);
      S   : out std_logic
   );
end mux;
```

```
architecture comportement of mux is
begin
   process(E, SEL)
      variable x : integer;
   begin
      S <= '0';  -- valeur initiale
      x := to_integer(unsigned(SEL));

      for i in 0 to 3 loop
         if i = x then
            S <= E(i);
         end if;
      end loop;
   end process;
end comportement;
```

6- Domaine séquentiel

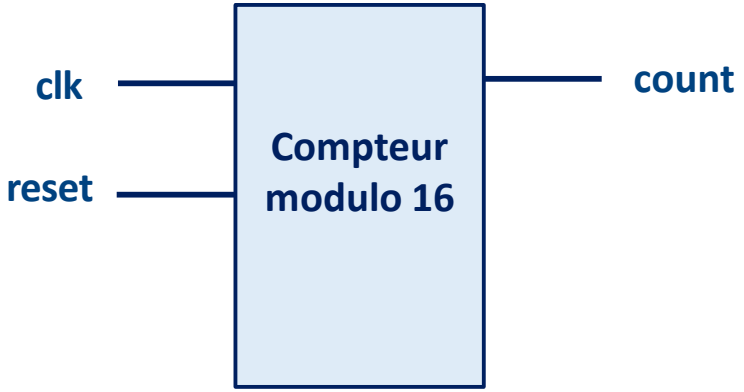
- Exemples de description VHDL des circuits séquentiels par utilisation de process
- Exemple : description VHDL d'une bascule D

- Le signal d'horloge CLK fait partie de la liste de sensibilité.
- Un changement d'état du signal CLK va déclencher le process et par conséquent évaluer les instructions de celui-ci.
- L'instruction « **rising_edge(CLK)** » permet de détecter un front montant du signal CLK. On peut utiliser aussi l'instruction « **CLK'event and CLK='1'** »
- Pour détecter un front descendant, on utilise l'instruction « **falling_edge(CLK)** » ou l'instruction « **CLK'event and CLK='0'** »

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity Bascule_d is
   Port (
      D , CLK : in  STD_LOGIC;
       Q   : out STD_LOGIC
   );
end Bascule_d;
architecture description of Bascule_d is
begin
   process(CLK)
   begin
      if rising_edge(CLK) then
         Q <= D;
      end if;
   end process;
end description;
```

6- Domaine séquentiel

- Exemples de description VHDL des circuits séquentiels par utilisation de process
- Exemple : description VHDL d'un compteur modulo 16




```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity compteur_modulo16 is
   Port (
      clk, reset  : in  STD_LOGIC;
     Q : out STD_LOGIC_VECTOR(3 downto 0) );
end compteur_modulo16;
```

6- Domaine séquentiel

- Exemples de description VHDL des circuits séquentiels par utilisation de process
- Exemple : description VHDL d'un compteur modulo 16

```
architecture description of compteur_modulo16 is
   signal compteur : unsigned(3 downto 0) := (others => '0');
begin
   process(clk, reset)
   begin
      if reset = '1' then
         compteur <= (others => '0');
elsif rising_edge(clk) then
         if compteur = "1111" then
            compteur <= ''0000''; -- ou bien compteur <=  (others => '0');
         else
            compteur <= compteur + 1;
         end if;
      end if;
   end process;
   Q <= std_logic_vector(compteur);  -- Conversion en std_logic_vector
end description;
```

```
if compteur = "1111" then
            compteur <= ''0000'';
else
```

Cette partie est facultative dans le cas d'un modulo 16, puisque Q est un vecteur de 4 bits, la valeur maximale est ''1111'', on ne peut pas la dépasser.

Cette partie devient importante dans le cas d'un compteur à 4 bits et de modulo inférieur à 16. Par exemple modulo 12. Dans ce cas la, il faut écrire:

```
if compteur = "1011" then
            compteur <= ''0000'';
else
```

7- Différentes descriptions VHDL

- En modélisation VHDL, on trouve trois principales types de descriptions: flot de données, comportementale et structurelle.

❑ Description flot de données

- Description de la manière dont les données circulent de signal en signal, ou d'une entrée vers une sortie.
- Utiliser des instructions concurrentes d'assignation de signal.
- Trois types d'instructions :
  - ➢ Affectation simple, description par équations logiques,
  - ➢ when…else, with…select

❑ Description comportementale

- La description comportementale décrit le fonctionnement d'un circuit en se basant sur sa fonctionnalité plutôt que sur sa structure interne.
- L'architecture est décrite comme une suite d'instructions, comme en langages de programmation traditionnels.
- Elle fait appel à des process.

7- Différentes descriptions VHDL

❑ Description structurelle

- Pour instancier (interconnecter) les composants dans une description structurelle, on utilise le mot clé " port map" comme suit:
- Pour declarer un composant, on utilise le mot clé "component" comme suit:

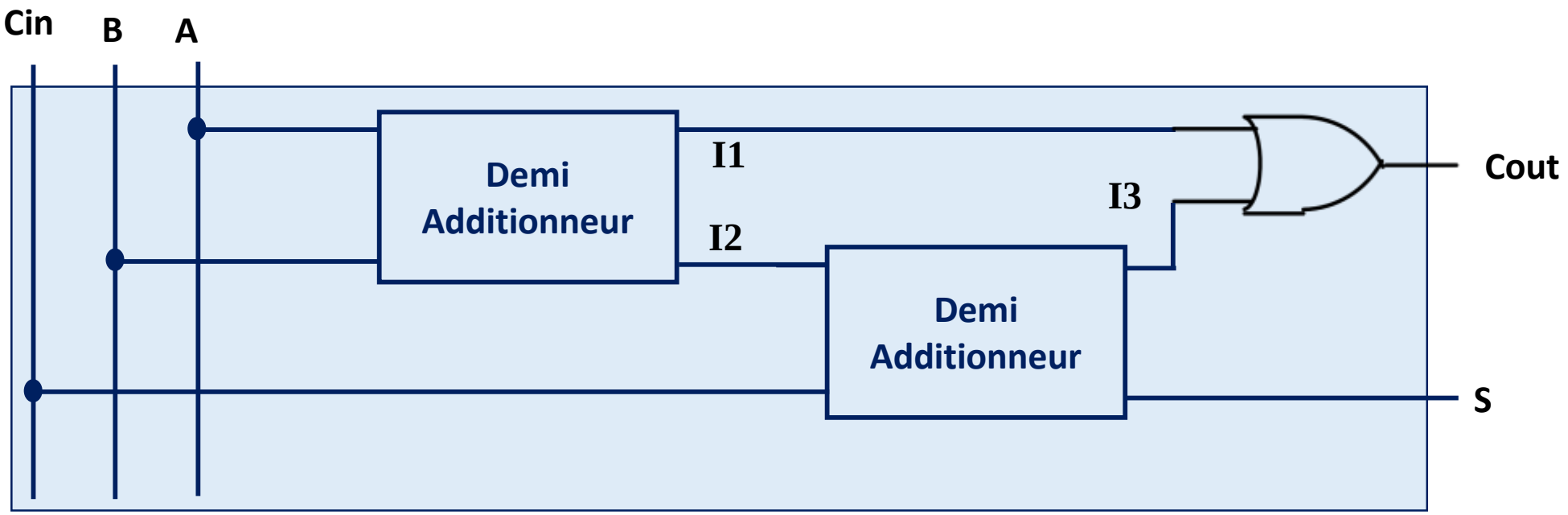
```
component Nom_ component
port (
description des signaux);
end component;
```

- La description structurelle permet d'assembler plusieurs blocs fonctionnel (composants) écrits en VHDL.

```
<NOM_INSTANCE>:<NOM_COMPOSANT> port map (LISTE DES CONNEXIONS);
```

7- Différentes descriptions VHDL

❑ Exemple de Description structurelle d'un additionneur complet avec des demi-additionneurs



7- Différentes descriptions VHDL

- Exemple de Description structurelle d'un additionneur complet avec des demi-additionneurs

VHDL porte OR

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity porte_or is
   port(
      A, B : in  std_logic;
         S    : out std_logic
   );
end porte_or;

architecture description of porte_or is
begin
   S <= A or B;
end description;
```

VHDL demi-additionneur

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity demi_add is
   port(
      A, B : in  std_logic;
      Cin    : out std_logic;
      S    : out std_logic
   );
end demi_add;

architecture description of demi_add is
begin
   S <= A xor B;
   Cin <= A and B;
end description;
```

7- Différentes descriptions VHDL

Description structurelle VHDL additionneur complet

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity add_complet is
   port(
      A, B, Cin : in  std_logic;
      S, Cout   : out std_logic
   );
end add_complet ;
architecture structural of add_complet  is
   component demi_add
      port(
         A, B : in  std_logic;
         Cin    : out std_logic;
         S    : out std_logic
      );
   end component;
   component porte_or
      port(
         X, Y : in  std_logic;
         Z    : out std_logic
      );
   end component;
```

Déclaration des composants

Déclaration des signaux internes

```
signal I1, I2, I3 : std_logic;
```

```
Begin

Ad1 : demi_add  port map(A, B, I1, I2);
Ad2 : demi_add port map(I2, Cin, I3, S);
OR1 : porte_or port map(I1, I3, Cout);
end structural;
```

Instanciation des composants